FUNDAMENTALS OF ARTIFICIAL INTELLIGENCE

INTELLIGENT TRAFFIC SIGNAL CONTROL USING FUZZY LOGIC

A simulation-based case study of adaptive green-light timing and priority-aware signal scheduling

Project type	AI case study and interactive Python simulation
Core concept	Fuzzy logic, membership functions, and rule-based inference
Scheduling	One green approach at a time, with service for all four directions
Implementation	Python 3 and Tkinter GUI

Prepared for academic demonstration. This is a simulation and must not be connected to real-world traffic signals.

TABLE OF CONTENTS

1.	Abstract
2.	Introduction
3.	Problem Statement and Objectives
4.	Fuzzy Logic: Theoretical Background
5.	System Design and Architecture
6.	Fuzzy Inputs, Membership Functions and Rule Base
7.	Green-Time Calculation
8.	Priority Scheduling and Signal Sequence
9.	Application Workflow and User Interface
10.	Implementation and Testing
11.	Results, Limitations and Future Enhancements
12.	Conclusion
13.	References
Appendix A	Complete Python Source Code

The source code is reproduced at the end of this report.

1. Abstract

Traffic congestion at road intersections can cause delays, long queues, and inefficient use of road capacity. Fixed-time traffic signals allocate a predetermined green duration to each approach even when traffic conditions vary. This case study presents an Intelligent Traffic Signal Control system that uses fuzzy logic to estimate an appropriate green-light duration from two inputs: the number of waiting vehicles and their waiting time. The application classifies these inputs into linguistic categories, evaluates fuzzy IF–THEN rules, and combines their outputs into one suggested duration. A separate scheduler selects which approach receives green next and ensures that North, South, East, and West each receive a turn in every cycle. The Python desktop simulation displays the intersection, signal indicators, countdown timer, and rule-inspection panel. It demonstrates fuzzy reasoning without machine learning or external libraries. It is an educational prototype and is not designed to control actual traffic infrastructure.

2. Introduction

Artificial Intelligence includes methods that help systems make decisions under changing or uncertain conditions. Fuzzy logic is useful when inputs do not fit strict categories. For example, traffic density can be described as low, medium, or high, with partial membership in more than one category. This project models a four-way intersection and uses vehicle count and waiting time to calculate variable green durations. Fuzzy logic answers “how long?”, while the scheduler answers “which road next?”.

3. Problem Statement and Objectives

Problem statement: A fixed-duration signal does not adapt its green time to changing queue sizes and waiting times. The project demonstrates an AI-based controller that calculates variable green durations and schedules four approaches without allowing conflicting green signals at the same time.

Objectives

- Use vehicle count and waiting time as inputs for each approach.
- Represent density as low, medium, and high using fuzzy membership functions.
- Represent waiting time as short, medium, and long.
- Apply a nine-rule fuzzy rule base to estimate green duration.
- Keep one approach green at a time and the other three red.
- Serve each approach once per cycle using priority-aware scheduling.
- Provide a GUI for generating traffic, running the controller, and inspecting rule activation.

4. Fuzzy Logic: Theoretical Background

Boolean logic uses crisp values such as true or false. Fuzzy logic allows partial membership between 0 and 1. A membership value of 0 means no membership in a category, 1 means full membership, and a value between them means partial membership. For example, a traffic count may belong to the medium category with membership 0.7 and the high category with membership 0.2.

A fuzzy inference system has four stages:

- **Fuzzification:** Convert numerical inputs into membership values for linguistic categories.
- **Rule evaluation:** Evaluate IF–THEN rules. Fuzzy AND is calculated using the minimum of the input memberships.
- **Aggregation:** Combine the contributions of rules whose strengths are greater than zero.
- **Defuzzification:** Convert the rule outputs into a numerical green-time value using a weighted average.

This implementation uses manually defined membership functions and rule outputs. It does not learn from historical traffic data.

5. System Design and Architecture

The application contains a traffic generator, fuzzy controller, scheduler, timer, and graphical interface. The traffic generator creates simulated vehicle counts and waiting times. The fuzzy controller calculates a suggested green duration for each approach. The scheduler selects the active approach and manages the countdown. The GUI displays the road layout, vehicle icons, signal colors, status, timer, and fuzzy-rule details.

Component	Responsibility
Traffic generator	Creates simulated vehicle counts and waiting-time values.
Fuzzy inference	Calculates membership values, rule strengths, and suggested green duration.
Scheduler	Chooses the next eligible approach; prioritizes longer waiting time and uses vehicle count to break ties.
Timer	Counts down the active green duration and triggers a switch.
GUI	Displays signals, traffic, status, priority summary, and selected-approach details.

6. Fuzzy Inputs, Membership Functions and Rule Base

The controller uses two inputs: vehicle count and waiting time. Vehicle count is classified as low, medium, or high density. Waiting time is classified as short, medium, or long. Membership functions overlap, so values near category boundaries can partly belong to multiple categories.

Density categories

- **Low density:** membership decreases from 1 to 0 as vehicle count moves from approximately 3 to 9.
- **Medium density:** triangular membership, peaking around 10 vehicles and extending approximately from 4 to 16.
- **High density:** membership increases from 0 to 1 as vehicle count moves from approximately 12 to 19.

Waiting-time categories

- **Short wait:** membership decreases from 1 to 0 between approximately 15 and 45 seconds.
- **Medium wait:** triangular membership, peaking around 60 seconds and extending approximately from 20 to 100 seconds.
- **Long wait:** membership increases from 0 to 1 between approximately 75 and 110 seconds.

Nine fuzzy IF–THEN rules

Density	Waiting category	Rule output
Low	Short	12 s
Low	Medium	18 s
Low	Long	28 s
Medium	Short	22 s
Medium	Medium	32 s
Medium	Long	42 s
High	Short	35 s
High	Medium	45 s
High	Long	58 s

7. Green-Time Calculation

For each rule, the strength is the minimum of the density membership and waiting-time membership. If w■ is the strength of rule i and t■ is its output duration, the controller uses the weighted average:

```
T = Σ(w■ × t■) / Σw■
```

The result is limited to 10–60 seconds. The suggested output may include decimals, while the countdown uses the nearest whole second. For example, medium density = 1.00 and short wait = 1.00 activates the Medium AND Short rule with strength min(1.00, 1.00) = 1.00. If it is the only active rule, the result is 22 seconds.

If medium/short fires with strength 0.43 and high/short fires with strength 0.33, the output is [(0.43 × 22) + (0.33 × 35)] / (0.43 + 0.33) ≈ 27.6 seconds, rounded to 28 seconds.

8. Priority Scheduling and Signal Sequence

Fuzzy logic calculates how long each approach should receive green; a separate scheduler determines which approach goes next. The scheduler considers approaches not yet served in the current cycle and selects the one with the greatest waiting-time value. If waiting times are equal, vehicle count breaks the tie. After all four approaches have received green, a new cycle begins.

At any moment, only the active approach is green; the other three remain red. When the timer reaches zero, the current approach is marked as served and the scheduler activates the next eligible approach.

Simulation limitation: waiting-time values are generated when new traffic is created and do not continuously increase while an approach waits. The demo is a simplified illustration of scheduling, not a validated traffic-control model.

9. Application Workflow and User Interface

- **New Traffic:** Generates new vehicle counts and waiting-time values, and stops the current cycle.
- **Calculate & Start / Run Fuzzy Controller:** Evaluates rules for each approach, calculates durations, selects the first approach, and starts the countdown.
- **Pause:** Stops the cycle timer.
- **Resume:** Continues a paused cycle.
- **Inspect approach:** Displays the selected approach’s vehicle count, waiting time, signal state, memberships, suggested duration, and active rules.

Green indicates the currently active approach; red indicates an approach that must wait. Blue rectangles represent simulated vehicles.

10. Implementation and Testing

The project is implemented in Python 3 with Tkinter and the standard library. The canvas draws the intersection, vehicle icons, labels, and signals. The after() callback updates the countdown without freezing the interface.

Suggested manual tests

Test	Action	Expected behavior
1	Click New Traffic	New vehicle counts and wait values appear; cycle stops.
2	Start the controller	Durations are calculated and exactly one signal is green.
3	Wait for timer to end	Current road turns red and another road becomes green.
4	Click Pause	Countdown stops.
5	Click Resume	Countdown continues.
6	Inspect each approach	Details show memberships and active rules.
7	Observe a full cycle	All four approaches are served before a new cycle begins.

11. Results, Limitations and Future Enhancements

The simulation demonstrates variable green-light durations based on traffic density and waiting-time categories. It also demonstrates sequential signal operation: one approach is green at a time while the other three remain red. The inspection panel makes decisions easier to explain by showing membership values and active rules.

Limitations

- Traffic values are simulated rather than measured by sensors or cameras.
- Waiting time does not automatically increase while a road is waiting.
- Membership boundaries and rule outputs are manually selected for demonstration.
- The model does not include turning lanes, pedestrian crossings, emergency vehicles, or clearance intervals.
- It has not been validated for real-world deployment.

Future enhancements

- Increase waiting time continuously for red approaches and update queues when vehicles pass.
- Add amber lights and all-red clearance intervals.
- Allow users to edit traffic values and membership-function thresholds.
- Compare fuzzy control with fixed-time control using average wait and queue length.
- Use a dedicated traffic simulator for more realistic experiments.

12. Conclusion

This case study presents an explainable AI application for adaptive traffic signal timing. Fuzzy logic maps vehicle count and waiting time into a suggested green duration through membership functions and IF–THEN rules. A separate scheduler determines which approach receives green and ensures every direction is served in a cycle. The interactive interface demonstrates fuzzification, rule activation, weighted output calculation, countdown timing, and sequential scheduling. The prototype is appropriate for an introductory AI case study, but not for controlling real traffic signals.

13. References

1. L. A. Zadeh, “Fuzzy Sets,” Information and Control, vol. 8, no. 3, pp. 338–353, 1965.

2. Timothy J. Ross, Fuzzy Logic with Engineering Applications, 3rd ed., Wiley, 2010.

3. Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” https://docs.python.org/3/library/tkinter.html.

4. Project implementation: fuzzy_traffic_signal.py, created for this educational case study.

APPENDIX A: COMPLETE PYTHON SOURCE CODE

The complete source code used for the demo is reproduced below. It is included as text so the program can be reviewed alongside the explanation.

```
"""
Intelligent Traffic Signal Control Using Fuzzy Logic — Sequential Signal Demo

Run:
    python fuzzy_traffic_signal.py

Requirements:
    Python 3.x with Tkinter. No pip packages required.

Features:
- Four simulated approaches: North, East, South, West.
- Fuzzy logic calculates a suggested green duration from vehicle count and waiting time.
- Only ONE approach is green at a time; the other three remain red.
- The active signal automatically switches when its countdown reaches zero.
- Every road gets a turn. The next road is selected using a fair, priority-aware
  scheduler: longest-waiting approaches are favored, then larger queues.
- New Traffic generates a fresh scenario. Recalculate restarts the schedule.
"""

import tkinter as tk
from tkinter import ttk
import random

GREEN_MIN, GREEN_MAX = 10, 60
TICK_MS = 1000

# ---------------- Fuzzy logic: membership functions ----------------

def triangular(x, a, b, c):
    if x &lt;= a or x &gt;= c:
        return 0.0
    if x == b:
        return 1.0
    if x &lt; b:
        return (x - a) / (b - a) if b != a else 1.0
    return (c - x) / (c - b) if c != b else 1.0

def shoulder_low(x, a, b):
    if x &lt;= a:
        return 1.0
    if x &gt;= b:
        return 0.0
    return (b - x) / (b - a)

def shoulder_high(x, a, b):
    if x &lt;= a:
        return 0.0
    if x &gt;= b:
        return 1.0
    return (x - a) / (b - a)

def fuzzify_density(count):
    # Overlapping sets: low, medium, high
    return {
        "low": shoulder_low(count, 3, 9),
        "medium": triangular(count, 4, 10, 16),
        "high": shoulder_high(count, 12, 19),
    }
```

```
def fuzzify_wait(wait):
    return {
        "short": shoulder_low(wait, 15, 45),
        "medium": triangular(wait, 20, 60, 100),
        "long": shoulder_high(wait, 75, 110),
    }

def fuzzy_green_time(vehicle_count, wait_seconds):
    """Evaluate fuzzy rules and return a bounded green duration in seconds."""
    d = fuzzify_density(vehicle_count)
    w = fuzzify_wait(wait_seconds)

    rules = [
        ("low", "short", 12),
        ("low", "medium", 18),
        ("low", "long", 28),
        ("medium", "short", 22),
        ("medium", "medium", 32),
        ("medium", "long", 42),
        ("high", "short", 35),
        ("high", "medium", 45),
        ("high", "long", 58),
    ]

    numerator = denominator = 0.0
    fired = []
    for density_label, wait_label, seconds in rules:
        strength = min(d[density_label], w[wait_label])  # fuzzy AND
        if strength &gt; 0:
            numerator += strength * seconds
            denominator += strength
            fired.append((density_label, wait_label, seconds, strength))

    # Fallback protects edge cases where no rule has nonzero strength.
    result = numerator / denominator if denominator else 20.0
    return max(GREEN_MIN, min(GREEN_MAX, result)), fired, d, w

# ---------------- GUI and sequential signal scheduling ----------------

class TrafficDemo:
    def __init__(self, root):
        self.root = root
        root.title("Fuzzy Logic Traffic Signal Controller — Sequential Mode")
        root.geometry("1120x800")
        root.minsize(900, 700)

        self.roads = ["North", "East", "South", "West"]
        self.counts = {}
        self.waits = {}
        self.results = {}
        self.green_seconds = {}
        self.current_road = None
        self.remaining = 0
        self.timer_running = False
        self.elapsed = 0
        self.last_served = []
        self.selected = tk.StringVar(value="North")
```

```
        header = ttk.Frame(root, padding=12)
        header.pack(fill="x")
        ttk.Label(header, text="Intelligent Traffic Signal Control Using Fuzzy Logic",
                  font=("Arial", 16, "bold")).pack(anchor="w")
        ttk.Label(header, text="One green signal at a time • adaptive duration • fair rotation",
                  font=("Arial", 10)).pack(anchor="w", pady=(4, 0))

        controls = ttk.Frame(root, padding=(12, 4))
        controls.pack(fill="x")
        ttk.Button(controls, text="New Traffic", command=self.new_traffic).pack(side="left", padx=(0, 8))
        ttk.Button(controls, text="Calculate &amp; Start", command=self.calculate_and_start).pack(side="left", padx=(0, 8))
        ttk.Button(controls, text="Pause", command=self.pause).pack(side="left", padx=(0, 8))
        ttk.Button(controls, text="Resume", command=self.resume).pack(side="left", padx=(0, 12))
        ttk.Label(controls, text="Inspect approach:").pack(side="left")
        combo = ttk.Combobox(controls, textvariable=self.selected, values=self.roads,
                             state="readonly", width=10)
        combo.pack(side="left", padx=6)
        combo.bind("&lt;&lt;ComboboxSelected&gt;&gt;", lambda _e: self.update_details())

        self.canvas = tk.Canvas(root, bg="#e8ecef", highlightthickness=0, height=440)
        self.canvas.pack(fill="both", expand=True, padx=12, pady=8)

        bottom = ttk.Frame(root, padding=12)
        bottom.pack(fill="x")
        self.status = ttk.Label(bottom, text="Click Calculate &amp; Start to calculate timings and start the signal cycle.",
                                font=("Arial", 10), wraplength=1050)
        self.status.pack(anchor="w")
        self.priority_label = ttk.Label(bottom, text="", font=("Arial", 10), wraplength=1050)
        self.priority_label.pack(anchor="w", pady=(4, 0))
        self.details = ttk.Label(bottom, text="", font=("Consolas", 9), justify="left", wraplength=1050)
        self.details.pack(anchor="w", pady=(8, 0))

        self.new_traffic()

    def new_traffic(self):
        self.timer_running = False
        self.current_road = None
        self.remaining = 0
        self.elapsed = 0
        self.last_served = []
        self.results = {}
        self.green_seconds = {}
        for road in self.roads:
            self.counts[road] = random.randint(0, 20)
            self.waits[road] = random.randint(0, 120)
        self.status.config(text="New traffic generated. Click Calculate &amp; Start to run the fuzzy controller.")
        self.priority_label.config(text="")
        self.draw()
        self.update_details()

    def calculate_and_start(self):
        self.timer_running = False
        self.results = {}
        self.green_seconds = {}
        for road in self.roads:
            secs, fired, d, w = fuzzy_green_time(self.counts[road], self.waits[road])
            self.results[road] = (secs, fired, d, w)
```

```
            self.green_seconds[road] = max(1, round(secs))
        self.last_served = []
        self.elapsed = 0
        self.current_road = self.choose_next_road()
        self.remaining = self.green_seconds[self.current_road]
        self.timer_running = True
        self.update_priority()
        self.draw()
        self.update_details()
        self.tick()

    def choose_next_road(self):
        # Fairness rule: roads not served in this cycle get priority first.
        # Among them, choose greatest waiting time; then vehicle count.
        candidates = [r for r in self.roads if r not in self.last_served]
        if not candidates:
            self.last_served = []
            candidates = self.roads[:]
        return max(candidates, key=lambda r: (self.waits[r], self.counts[r]))

    def update_priority(self):
        ranked = sorted(self.roads, key=lambda r: (self.waits[r], self.counts[r]), reverse=True)
        order = " → ".join(f"{r} ({self.green_seconds[r]}s)" for r in ranked)
        self.priority_label.config(text="Initial wait/queue priority (durations shown): " + order)

    def tick(self):
        if not self.timer_running:
            return
        self.draw()
        self.update_details()
        if self.remaining &lt;= 0:
            self.last_served.append(self.current_road)
            self.current_road = self.choose_next_road()
            self.remaining = self.green_seconds[self.current_road]
            self.status.config(text=f"Signal switched: {self.current_road} is GREEN for {self.remaining} seconds. All other app
            self.draw()
            self.update_details()
        else:
            self.status.config(text=f"ACTIVE SIGNAL: {self.current_road} is GREEN • {self.remaining} seconds remaining. All oth
        self.remaining -= 1
        self.root.after(TICK_MS, self.tick)

    def pause(self):
        self.timer_running = False
        self.status.config(text="Signal cycle paused. Click Resume to continue.")
        self.draw()

    def resume(self):
        if not self.results:
            self.calculate_and_start()
            return
        if not self.timer_running:
            self.timer_running = True
            self.status.config(text="Signal cycle resumed.")
            self.tick()

    def update_details(self):
        road = self.selected.get()
```

```
        if road not in self.counts:
            return
        d = fuzzify_density(self.counts[road])
        w = fuzzify_wait(self.waits[road])
        if road in self.results:
            secs, fired, _, _ = self.results[road]
            active = ", ".join(f"{a}/{b}→{g}s [{strength:.2f}]" for a,b,g,strength in fired)
            result = f"Suggested green: {secs:.1f}s; rounded timer: {self.green_seconds[road]}s\nRules: {active or 'fallback du
        else:
            result = "Suggested green: not calculated yet"
        signal = "GREEN" if road == self.current_road and self.timer_running else "RED"
        remaining = f"; remaining={self.remaining}s" if signal == "GREEN" else ""
        self.details.config(text=(
            f"{road}: vehicles={self.counts[road]}, waiting={self.waits[road]}s, signal={signal}{remaining}\n"
            f"Density membership: low={d['low']:.2f}, medium={d['medium']:.2f}, high={d['high']:.2f}\n"
            f"Waiting membership: short={w['short']:.2f}, medium={w['medium']:.2f}, long={w['long']:.2f}\n"
            f"{result}"
        ))

    def draw(self):
        c = self.canvas
        c.delete("all")
        c.update_idletasks()
        width, height = max(c.winfo_width(), 700), max(c.winfo_height(), 440)
        cx, cy, road_w = width / 2, height / 2, 150
        c.create_rectangle(cx-road_w/2, 0, cx+road_w/2, height, fill="#454b54", outline="")
        c.create_rectangle(0, cy-road_w/2, width, cy+road_w/2, fill="#454b54", outline="")
        for yy in range(0, int(height), 30):
            c.create_line(cx, yy, cx, yy+14, fill="white", width=2)
        for xx in range(0, int(width), 30):
            c.create_line(xx, cy, xx+14, cy, fill="white", width=2)

        positions = {
            "North": (cx, cy-road_w/2-70),
            "South": (cx, cy+road_w/2+65),
            "West": (cx-road_w/2-100, cy),
            "East": (cx+road_w/2+100, cy),
        }
        for road in self.roads:
            px, py = positions[road]
            count, wait = self.counts[road], self.waits[road]
            c.create_text(px, py-35, text=f"{road}: {count} vehicles", font=("Arial", 10, "bold"), fill="#17202a")
            c.create_text(px, py-17, text=f"Wait: {wait}s", font=("Arial", 9), fill="#17202a")

            # Draw up to 10 car icons, for visual clarity.
            for i in range(min(count, 10)):
                if road in ("North", "South"):
                    vx = cx - 44 + (i % 2) * 26
                    vy = (cy-road_w/2-27-(i//2)*17) if road == "North" else (cy+road_w/2+7+(i//2)*17)
                else:
                    vx = (cx-road_w/2-27-(i//2)*26) if road == "West" else (cx+road_w/2+7+(i//2)*26)
                    vy = cy - 40 + (i % 2) * 26
                c.create_rectangle(vx-8, vy-5, vx+8, vy+5, fill="#4c9be8", outline="#173f5f")

            is_green = road == self.current_road and self.timer_running
            color = "#20a464" if is_green else "#c94b4b"
            label = f"GREEN\n{max(0,self.remaining)}s" if is_green else "RED"
            c.create_oval(px-23, py+5, px+23, py+51, fill=color, outline="white", width=2)
```

```
            c.create_text(px, py+28, text=label, fill="white", font=("Arial", 8, "bold"), justify="center")

        c.create_text(cx, cy, text="INTERSECTION", fill="white", font=("Arial", 10, "bold"))

if __name__ == "__main__":
    root = tk.Tk()
    app = TrafficDemo(root)
    root.mainloop()
```